

Travaux Pratiques : Analyse numérique

Master Matière et Rayonnement

Réaliser par :

**RADIA RAGUIQUE
RAJAA SALIH**

TP N°1 : Calcule numérique

But du TP :

L'objectif de ce travail pratique était de concevoir et d'implémenter des opérations arithmétiques sur des nombres en calcul flottant en base décimale, en utilisant le langage de programmation C.

A. Déclaration de la structure FLOT :

Une structure nommée FLOT a été déclarée, incluant deux membres : la mantisse (m) représentant la partie significative du nombre flottant, et l'exposant (e) indiquant la puissance de 10 à appliquer pour obtenir la valeur réelle.

- **Algorithme :**

```
Algorithme StructureFlot  
Déclaration de la structure FLOT  
m : réel  
e : entier  
Fin Déclaration  
Fin Algorithme
```

- **Code en C :**

```
1. #include <stdio.h>  
2.  
3.  
4. typedef struct {  
5.     float m;  
6.     int e;  
7. } FLOT;
```

B. Fonction FLOTTANT :

La fonction FLOTTANT a été implémentée pour convertir un nombre réel en simple précision en sa représentation flottante de type FLOT.

- **Algorithme :**

```
Entrées : m,e
Sortie : la valeur de x
Début :
    Si m < 1 Alors :
        Tanque m < 0.09 Faire :
            m ← m×10
            e ← e - 1
        Fin tanque.
    Sinon m ≥ 1 :
        m ←  $\frac{m}{10}$ 
        e ← e + 1
    Fin Si.
Fin.
```

- **Code en C :**

```
1.  #include <stdio.h>
2.
3.
4.  typedef struct {
5.      float m;
6.      int e;
7.  } FLOT;
8.
9.  FLOT FLOTTANT(float m, int e) {
10.      FLOT x;
11.      x.m = m;
12.      x.e = e;
13.      if (x.m < 1) {
14.          while (x.m < 0.09) {
15.              x.m = x.m * 10;
16.              x.e = x.e - 1;
17.          } return x;
18.
19.      } else {
20.          while (x.m >= 1) {
21.              x.m = x.m / 10;
22.              x.e = x.e + 1;
23.          } return x;}}
```

C. Fonction SOMMEFL :

Une fonction SOMMEFL est à implémenter en C, prenant en argument deux nombres flottants (pouvant avoir des mantisses de longueurs différentes) et retournant leur somme flottante.

- **Algorithme :**

```
Entrées : m1, e1, m2, e2
Sortie : la valeur de S
Début :
    Si e1 = e2 Alors
        m ← m1 + m2
        e ← e1
    Si e1 > e2 Alors
        t ← e1 - e2
        m1 ← m1 × 10t
        m ← m1 + m2
        e ← e2
    Sinon
        t ← e2 - e1
        m2 ← m2 × 10t
        m ← m1 + m2
        e ← e1
    Fin Si
    S ← FLOTTANT(m, e)
Fin
```

- **Code en C :**

```
1. FLOT SOMMEFL(float m1, int e1, float m2, int e2) {
2.     FLOT s;
3.
4.     if (e1 == e2) {
5.         s.m = m1 + m2;
6.         s.e = e1;
7.     }else if (e1 > e2) {
8.         int t = e1 - e2;
9.         m1 *= pow(10, t);
10.        s.m = m1 + m2;
11.        s.e = e2;
12.    } else {
13.        int t = e2 - e1;
14.        m2 *= pow(10, t);
15.        s.m = m1 + m2;
16.        s.e = e1;
17.    s=FLOTTANT(s.m, s.e);
18.    return s;}
```

D. Fonction DIFFL :

Une autre fonction DIFFL est à écrire en C, prenant deux nombres flottants et retournant leur différence flottante en utilisant la fonction SOMMEFL précédemment créée.

- **Algorithme :**

```
Entrées : m1, e1, m2, e2
Sortie : la valeur de DIF
Début :
  Si e1 = e2 Alors
    m ← m1 - m2
    e ← e1
  Si e1 > e2 Alors
    t ← e1 - e2
    m1 ← m1 ×  $\llbracket 10 \rrbracket^t$ 
    m ← m1 - m2
    e ← e2
  Sinon
    t ← e2 - e1
    m2 ← m2 ×  $\llbracket 10 \rrbracket^t$ 
    m ← m1 - m2
    e ← e1
  Fin Si
  DIF ← FLOTTANT(m, e)
Fin
```

- **Code en C :**

```
1. FLOT DIFFL(float m1, int e1, float m2, int e2) {
2.     FLOT dif;
3.
4.     if (e1 == e2) {
5.         dif.m = m1 - m2;
6.         dif.e = e1;
7.     } else if (e1 > e2) {
8.         int t = e1 - e2;
9.         m1 *= pow(10, t);
10.        dif.m = m1 - m2;
11.        dif.e = e2;
12.    } else {
13.        int t = e2 - e1;
14.        m2 *= pow(10, t);
15.        dif.m = m1 - m2;
16.        dif.e = e1;
17.    }
18.    dif=FLOTTANT(dif.m, dif.e);
19.    return dif;}
```

E. Fonction MULFL :

Création d'une fonction MULFL qui, à partir de deux nombres flottants, retourne leur produit flottant.

- **Algorithme :**

```
Entrées : m1, e1, m2, e2
Sortie : la valeur de M
Début
  Si m1 = 0 ou m2 = 0 Alors
    M ← M = 0
  Sinon
    m ← m1 × m2
    e ← e1 + e2
    Fin Si
    M ← FLOTTANT(m, e)
Fin
```

- **Code en C :**

```
1. FLOT MULFL(float m1, int e1, float m2, int e2) {
2.     FLOT M;
3.     if (m1==0 || m2==0 )
4.     {printf("\n"); }
5.     else {
6.         M.m = m1 * m2;
7.         M.e = e1 + e2;
8.
9.     M=FLOTTANT (M.m, M.e);
10.    return M;}}
```

F. Fonction DIVFL :

Implémentation d'une fonction DIVFL pour effectuer la division de deux nombres flottants en utilisant la fonction MULFL.

- **Algorithme :**

```
Entrées : m1, e1, m2, e2
Sortie : la valeur de DIV
Début
  Si m2 = 0 Imprimer « erreur »
  Sinon
    m ← m1 / m2
    e ← e1 - e2
  Fin Si
  div ← FLOTTANT(m, e)
Fin
```

- **Code en C :**

```

1. FLOT DIVFL(float m1, int e1, float m2, int e2) {
2.     FLOT div;
3.     if (m2==0 ){printf("Opération invalide\n");}
4.     else {
5.         div.m = m1 / m2;
6.         div.e = e1 - e2;
7.
8.     div=FLOTTANT(div.m, div.e);
9.     return div;}}

```

G. Fonction main :

La fonction principale (main) a été développée pour gérer l'entrée de deux nombres réels et d'un opérateur arithmétique. En fonction de l'opérateur saisi, l'opération correspondante sur les nombres flottants est effectuée, et le résultat est affiché à l'écran. Des mécanismes de gestion des exceptions ont été inclus pour gérer les cas où les opérations ne peuvent être effectuées correctement.

- **Algorithme :**

```

Entrées : n1, n2, operateur
Sortie : la valeur RESULTAT
Début :
    Si l'opérateur = + :
        RESULTAT ← SOMMEFL(m,e)
    Si l'opérateur = - :
        RESULTAT ← DIFFL(m,e)
    Si l'opérateur = ×:
        RESULTAT ← MULFL(m,e)
    Si l'opérateur = / :
        RESULTAT ← DIVFL(m,e)
    Sinon ( autres operateurs )
        Imprimer « Opérateur invalide »
Fin

```

- **Code en C :**

```

1. int main() {
2.     FLOT n1, n2, resultat;
3.     char operateur;
4.
5.     printf("Entrez m1 et e1 pour le nombre n1: ");
6.     scanf("%f %d", &n1.m, &n1.e);
7.     printf("Entrez m2 et e2 pour le nombre n2: ");
8.     scanf("%f %d", &n2.m, &n2.e);
9.
10.    printf("Entrez l'opérateur (+, -, *, /) : ");
11.    scanf(" %c", &operateur);
12.
13.
14.    switch (operateur) {
15.        case '+':
16.            resultat = SOMMEFL(n1.m, n1.e, n2.m, n2.e);
17.            break;
18.        case '-':
19.            resultat = DIFFL(n1.m, n1.e, n2.m, n2.e);
20.            break;
21.        case '*':
22.            resultat = MULFL(n1.m, n1.e, n2.m, n2.e);
23.            break;
24.        case '/':
25.            resultat = DIVFL(n1.m, n1.e, n2.m, n2.e);
26.            break;
27.        default:
28.            printf("Opérateur invalide\n");
29.            return 1;
30.    }
31.    printf("Le résultat est:%f10^%d\n", resultat.m, resultat.e);
32.
33.    return 0;}

```

Conclusion :

Ce travail pratique a permis de se familiariser avec la représentation des nombres flottants en base décimale et les opérations arithmétiques associées. Il a également fourni une compréhension pratique des défis liés à la précision et à la manipulation des nombres flottants lors de l'exécution d'opérations mathématiques.

La réalisation de ce travail a renforcé la compréhension des structures de données, des fonctions et des mécanismes de gestion des erreurs en programmation en langage C.

TP N°2 : Résolution de $f(x)=0$

But du TP :

Ce travail pratique avait pour but d'étudier et d'appliquer différentes méthodes numériques pour résoudre numériquement des équations non linéaires de la forme $f(x)=0$. Les méthodes de la bisection, de Newton-Raphson, de la sécante et de point fixe ont été mises en œuvre en langage C pour estimer les solutions de telles équations.

I. Méthode de la Bisection :

Cette méthode divise continuellement l'intervalle initial où la fonction change de signe par deux, jusqu'à ce qu'une racine soit trouvée avec une précision donnée.

- **Algorithme :**

- Démarre avec un intervalle $[a, b]$ tel que $f(a) * f(b) < 0$ (la fonction change de signe entre a et b).
- Calcule le point milieu $c = (a + b) / 2$.
- Si $f(c)$ est suffisamment proche de zéro (c'est-à-dire, $|f(c)| < \text{précision}$), c est la racine. Sinon, remplace a ou b par c selon la valeur de $f(c)$ pour maintenir l'intervalle où la racine réside

Si $f(a) * f(c) < 0$ alors $c \rightarrow b=c$

Si $f(a) * f(c) > 0$ alors $b \rightarrow a=c$

- répète le processus jusqu'à la convergence.

```
Tant que (abs (b-a) > epsilon) % test d'arrêt
    m= (a+b)/2
    si (f(a) * f(m)) < 0
        b=m % s est dans [a,m]
    sinon
        a = m % s est dans [m,b]
    Fin si
Fin tant que
```

- **Code en C**

Pour résoudre l'équation $2\sin(x)-x=0$ dans l'intervalle $[1, 6]$.

```
1. include <stdio.h>
2. #include <math.h>
3.
4. int main() {
5.     double a = 1, b = 6;
6.     double fa = 2 * sin(a) - a;
```



```

7.     double fb = 2 * sin(b) - b;
8.     double eps = 0.0001;
9.     int nb = 0;
10.    printf("nb | x | f(x) \n");
11.    printf("-----\n");
12.    if (fa * fb < 0) {
13.        while ((b - a) > eps) {
14.            nb++;
15.            double x = (a + b) / 2;
16.            double fx = 2 * sin(x) - x;
17.
18.            if (fa * fx < 0) {
19.                b = x;
20.            } else {
21.                a = x;
22.            }
23.            printf("%d | %f | %f \n", nb, x, fx);
24.        }
25.    }
26.
27.    return 0;

```

II. Méthode de Newton-Raphson :

Cette méthode utilise la pente de la fonction pour estimer la racine en itérant à partir d'un point initial.

- **Algorithme :**

- Entrer la formule de f.
- Entrer la formule de sa dérivée.
- Donner une solution initiale x_0 qui doit être proche de la solution recherchée. Dans cette étape on peut tracer la courbe de f pour donner une estimation de la solution initiale x_0 .
- Calculer les termes : $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$
- Répéter la 4eme étape, jusqu'à la stabilisation du processus avec la précision demandée

- **Code en C :**

Voici le code en C utilisant la méthode de Newton-Raphson pour résoudre l'équation $2\sin(x) - x = 0$

```

1. #include <stdio.h>
2. #include <math.h>
3.
4. double f(double x) {
5.     return 2 * sin(x) - x;
6. }
7. double f1(double x) {
8.     return 2 * cos(x) - 1;
9. }

```

```

10.     int main() {
11.         double x0 = 1.0;
12.         double x;
13.         double precision = 1e-6;
14.         int max_iterations = 100;
15.
16.         int n = 0;
17.         while (n < max_iterations) {
18.             x = x0 - (f(x0) / f1(x0));
19.
20.             if (fabs(f(x)) < precision) {
21.                 printf("La solution approchée est : %lf\n", x);
22.                 break;}
23.
24.             x0 = x;
25.             n++;}
26.
27.         if (n == max_iterations) {
28.             printf("La méthode n'a pas convergé dans le nombre
d'itérations spécifié.\n");
29.         }
30.
31.         return 0;
32.     }

```

III. Méthode de Sécante :

Cette méthode est similaire à la méthode de Newton-Raphson mais au lieu d'utiliser la dérivée, elle approxime la pente à partir de deux points successifs.

- **Algorithme :**

```

Entree :  $X_0, X_1, n, \varepsilon$  (la précision désirée)
Sortie : la valeur approchée  $x : f(x)=0$ , ou message d'erreur
Début :
     $i \leftarrow 0$ 
     $Y_0 \leftarrow f(X_0)$ 
     $Y_1 \leftarrow f(X_1)$ 
    Tant que :  $i < n$  faire :
         $X = X_1 - Y_1 \frac{X_1 - X_0}{Y_1 - Y_0}$ 
        Si ( $|X_1 - X_0| \leq \varepsilon$ ) alors imprimer  $x$ , aller à fin
             $i \leftarrow i + 1$ 
             $X_0 \leftarrow X_1$ 
             $Y_0 \leftarrow Y_1$ 
             $X_1 \leftarrow X$ 
             $Y_1 \leftarrow f(x)$ 
        Fin si.
    Fin tant que.
    Imprimer la méthode a échoué après  $n$  itérations
Fin.

```

- **Code en C :**

Voici le code en langage C utilisant la méthode de la sécante pour résoudre l'équation $2\sin(x)-x=0$

```
1. #include <stdio.h>
2. #include <math.h>
3.
4. double f(double x) {
5.     return 2 * sin(x) - x;}
6.
7. int main() {
8.     double x0 = 1.0;
9.     double x1 = 2.0;
10.    double x;
11.    double precision = 1e-6;
12.    int max_iterations = 100;
13.
14.    int n = 0;
15.    while (n < max_iterations) {
16.        x = x1 - (f(x1) * (x1 - x0)) / (f(x1) - f(x0));
17.
18.        if (fabs(f(x)) < precision) {
19.            printf("La solution approchée est : %lf\n", x);
20.            break;}
21.
22.        x0 = x1;
23.        x1 = x;
24.        n++;}
25.
26.    if (n == max_iterations) {
27.        printf("La méthode n'a pas convergé dans le
nombre d'itérations spécifié.\n");}
28.
29.    return 0;
30. }
```

IV. Méthode du Point Fixe :

Cette méthode reformule l'équation $f(x)=0$ sous forme de $g(x)=x$ et utilise une fonction auxiliaire $g(x)$ pour itérer jusqu'à une estimation de la racine.

- **Algorithme :**

Entrées : x_0 , ε (la précision désirée), n
 Sortie : la valeur approchée x ou message d'erreur
 Début :
 $i \leftarrow 0$
 Tan que $i < n$ faire :
 $x \leftarrow g(x_0)$
 Si $(|x - x_0| \leq \varepsilon)$ alors imprimer x, aller à fin
 $i \leftarrow i+1$, $x_0 \leftarrow x$
 Fin si.
 Fin tant que.
 Imprimer la méthode a échoué après n itérations.
 Fin.

- **Code en C :**

Pour résoudre l'équation $2\sin(x) - x = 0$ en utilisant la méthode du point fixe, nous pouvons réécrire cette équation sous la forme $g(x) = x$ où $g(x) = 2\sin(x)$

```

1. #include <stdio.h>
2. #include <math.h>
3.
4. double g(double x) {
5.     return 2 * sin(x);
6. }
7. int main() {
8.     double x0 = 1.0;
9.     double x;
10.    double precision = 1e-6;
11.    int max_iterations = 100;
12.    int n = 0;
13.    while (n < max_iterations) {
14.        x = g(x0);
15.        if (fabs(x - x0) < precision) {
16.            printf("La solution approchée est : %lf\n", x);
17.            break;
18.        }
19.        x0 = x;
20.        n++;
21.    }
22.    if (n == max_iterations) {
23.        printf("La méthode n'a pas convergé dans le nombre
    d'itérations spécifié.\n");
24.    }
25.    return 0;

```

Conclusion :

Ce travail pratique a permis de comprendre et d'appliquer des méthodes numériques pour résoudre des équations non linéaires $f(x)=0$. Il a également permis de comparer différentes

approches, d'analyser la convergence et la précision de chaque méthode, et d'évaluer leur applicabilité pour différents types de fonctions.

La réalisation de ces méthodes numériques a fourni une expérience pratique dans la résolution numérique d'équations non linéaires, renforçant ainsi la compréhension des concepts de convergence, de stabilité et de précision des méthodes numériques en mathématiques appliquées. Ce travail a également souligné l'importance de la sélection de la méthode appropriée en fonction de la nature de l'équation et des conditions initiales pour obtenir des résultats fiables et précis.